

Классическая работа

Труднорешаемые задачи комбинаторной оптимизации

Задача называется труднорешаемой, если для ее решения не существует алгоритма полиномиальной сложности (полиномиального алгоритма). $O(n^p)$ - оценка

Алгоритмическая сложность

Класс NP - класс задач распознавания, в котором можно проверить ответ „ДА“ за полиномиальное время

Класс NP -полных задач - самые трудные задачи в NP ; если существует точный полиномиальный алгоритм для решения одной из них, то существует точный полиномиальный алгоритм и для других задач из класса NP

Класс NP -трудных задач - это задачи, которые могут не лежать в NP , но которые не проще NP -полных задач

Класс P - это класс задач распознавания, которые можно решить полиномиальным алгоритмом (Проблема века: $P=NP?$)

Задача распознавания свойств представляет собой задачу, решением которой является ответ „ДА“ или „НЕТ“. Эта задача состоит из двух частей: первая - что дано; вторая - вопрос, требующий ответа „ДА“ или „НЕТ“.

Задача распознавания свойств используется для исследования трудноре-
шаемых задач комбинаторной оптимизации.

Пример: задача Коши расхода (оптимизационная задача). Даны граничное
множество городов $C = \{c_1, c_2, \dots, c_n\}$. Расстояния $d(i, j) \in \mathbb{Z}^+$, $(i, j) \in C$.

• Найти: тур или маршрут оптимальной длины

Задача распознавания свойств (усложненная задача):

• Дано: $C = \{c_1, c_2, \dots, c_n\}$; $d \notin (i, j)$, $i, j = \overline{1, n}$; граница $B \in \mathbb{Z}^+$

• Вопрос: существует ли тур, проходящий через все города из C , длина которого
не превосходит B ?

Задача распознавания используется для определения класса задачи комбинаторной
оптимизации.

Классификация методов решения задач комбинаторной оптимизации

1. Точные: полный перебор; метод ветвей и границ

2. Приближенные: выделяют некоторое допустимое решение и обладают тем свойством,
что это решение близко к оптимальному; т.е. отличается от оптимального на
какую-то константу.

3. Эвристические: базируются на предположении строго предположении о (качестве)
свойствах оптимального решения задачи; эти методы включают приём, который
можно назвать „снижение требований к оптимальному решению“.

2 и 3 методы применяются для решения задач из класса NP.

1) Методы локального поиска

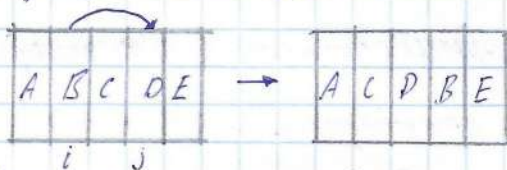
2) Конструктивные методы

Алгоритмы локального поиска ищут лучшее решение с некоторого начального решения и итеративно пытаются заменить текущее решение на лучшее в специальной определенной окрестности.

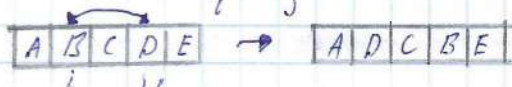
Полиномиальное время

Окрестность: множество решений $N(i_d)$, полученных путем суммирования перемешивания некоторой операции N к данному решению i_d позволяют окрестностью решений i_d .

Пример: операция сдвиг (i, j)



→ операция замена (i, j)



06.02 * Окрестности Лина-Кернигхана (на примере 3. Коммивояжера):

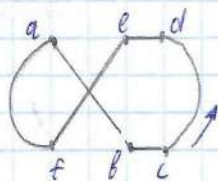
- окрестность 2-opt: пусть решение 3. Коммивояжера - это маршрут, который начинается из базового города и заканчивается в этом городе, причем из каждого города можно выбывать один раз. Совокупность маршрутов представляет собой Гамильтоновы циклы.

Окрестность - множество всех гамильтоновых циклов, получающихся из заданного цикла заменой двух несмежных ребер. Такая окрестность содержит $\frac{n(n-3)}{2}$ элементов

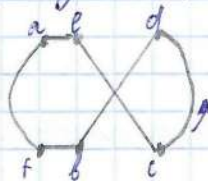
- окрестность на перестановках (city-swaps). пусть решение - маршрут - представляет собой перестановку $\pi = (i_1, i_2, \dots, i_n)$. Окрестность

$N(T_i)$ - множество ребер инцидентных, отличающихся от T_i только в узлах
 позиции их. Количество элементов $\frac{n(n-1)}{2}$

Различия между "2-opt" и "city-swap"



$a-b-c-d-e-f-a$

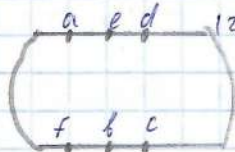


$a-e-c-d-b-f-a$

(city-swap)

• Если в "city-swap" поменяться местами "b" и "e", то

появятся 4 новых ребра $(ae), (bc), (bd), (cf)$; обход дуг останется прежним



Удаление ребер $(ab), (cd)$ приведет к появлению

новых (ae) и (bf) и обратный ход дуг от d и c;

$a-e-d-c-b-f-a$ можно расширить 2-opt до n-opt

• Радиус мультипликативной окрестности: $N(i)$, i - решение (окрестность),

$N_p(i) \subseteq N(i)$ - r-окрестность, строится следующим образом: каждый элемент $j \in N(i)$ включается в множество $N_p(i)$ с вероятностью p

• Окрестность экспоненциальной мощности: пусть множество городов и раз-

бито на две части: 1) $x_1, \dots, x_m$, 2) $y_1, \dots, y_k$; $k+m=n$, $m \leq k$. Добавим в первую часть $k-m$ фиктивных городов $x_{m+1}, \dots, x_n$ и рассмотрим окрестность

$N(y_1, \dots, y_k, x_1, \dots, x_m)$. состоящую из маршрутов вида $y_1 x_{t_1} y_2 x_{t_2} \dots y_k x_{t_k}$,

где $(t_1, \dots, t_k)$ - перестановка; т.е. каждый (новый) маршрут получается из старой

маршрута x_i между парами городов y_j и y_{j+1} . Окрестность $N(y_1, \dots, y_k, x_1, \dots, x_m)$

имет экспоненциальную мощность $\left[\frac{n+1}{2}\right]!$ при $2m \leq n \leq 2m+1$ $\lceil \cdot \rceil$ - округление

Общая схема алгоритма локального поиска: пусть S - решение, $N(S)$ - окрестность решений для S , $F(S)$ - целевая функция для S

Примеры окрестности: 1) замена двух компонентов решения (если это вектор или перестановка), мощность n^2 ;

2) перемещение компонентов; мощность n^2

Шаг 1: Выбрать начальное решение S и вычислить $F(S)$

Шаг 2: Найти в $N(S)$ решение S' с минимальным значением F :

$$F(S') = \min \{ F(S), S' \in N(S) \}$$

Шаг 3: Если $F(S') < F(S)$, то $S = S'$ и идти к "Шагу 2"

Общая схема конструктивного алгоритма:

Шаг 1: Упорядочить компоненты решения $k_i, i = \overline{1, n}$

Шаг 2: Пусть $S' = \emptyset, (S' \in S, S - \text{множество допустимых решений})$

Шаг 3: Повторять: если $S' \cup \{k_i\}$ допустимое частичное построение решения, то $S' = S' \cup \{k_i\}, i = i + 1$; пока $S' \in S$

Метакритерии:

1) эврика - идти, мета - лучше (или более высоким уровнем); разницу между эвристикой и метой алгоритма

2) в литературе не даётся строгое определение ^{типу} алгоритмов, а лишь перечисляются его основные свойства: - является недетерминированным;

- содержат процедуры, позволяющие выбирать из множества оптимальных;

- допускают абстрактный уровень описания, т.е. не являются проблемой ориентированности

Известные метаэвристики:

- | | |
|--------------------------------|------------------------------------|
| 1) генетический алгоритм | 5) поиск с запретами |
| 2) эволюционный алгоритм | 6) поиск с переменной окрестностью |
| 3) алгоритм муравьиной колонии | 7) вероятностно-жадный алгоритм |
| 4) имитация отжига | 8) нейронные сети |

Основные характеристики метаэвристик: каждая метаэвристика содержит две процедуры, которые называются: 1) процедура интенсификации поиска - т.е. процедура, позволяющая в пространстве поиска найти более качественное решение; 2) процедура диверсификации - позволяет искать решение в ещё не исследованных областях пространства поиска.

Генетический алгоритм: автор - Holland J.H. (1975)

Классная работа

13.02

Генетический алгоритм (ГА)

Общая схема

1. Выбрать начальную популяцию P и запомнить рекорд $F = \min_{i=1, \dots, N} f(S_i)$, где S_i - решение популяции, $f(S_i)$ - целевая ф-ция S_i . ($P = \{S_1, S_2, \dots, S_N\}$)
2. Пока не выполнены критерии остановки, выполнять:
 - 2.1. Выбрать "родителей" S_{i_1}, S_{i_2} из популяции P
 - 2.2. Применить к S_{i_1} и S_{i_2} оператор скрещивания и получить новое решение S'

2.3. Применить к S' оператор мутации и получить новое решение S''

2.4 (необязательный). Применить к S'' оператор локального улучшения решения и получить S'''

2.5. Если $f(S''') < F^*$, то сменить рекорд $F^* = f(S''')$;

придоговаривая, что решается задача на минимум

2.6. Добавить S''' к популяции P и удалить из нее худшие решения;

размер новой популяции должен соответствовать размеру исходной популяции.

Конец „пока“

Оператор „скрещивания“

S_1, S_2 - решения, компоненты этих решений 0 или 1

• односточечный оператор скрещивания: $S_1: (\underbrace{01001 \dots 011}_1); S_2: (\underbrace{11010 \dots 111}_2)$;

случайным образом находится позиция ℓ , $1 \leq \ell \leq n$: $S' (01000 \dots 111)$.

(используют двук-, трёх- и т.д. оператор скрещивания)

Выбор „родителей“ (селекция):

- турнирная селекция: из популяции P случайным образом выбирается некоторое количество $P' \subseteq P$ и родителями становятся наилучшее решение в P' :

$$S_i = \min_{S \in P'} f(S)$$

- пропорциональная селекция: из популяции P случайным образом выбирается два родителя. Для решения S_i вероятность быть выбранной обратно пропорциональна целевой ф-ии $f(S_i)$. Варианты селекции: лучший из популяции +

+ случайно выбранный; случайно выбранный + наиболее улучшенный от него изд.

Оператор "мутации": вероятностный оператор, который случайным образом вносит изменения в допустимое решение задачи. Пример: $S'(\overset{i_1}{0}1\overset{i_2}{0}0\dots011)$
случайным образом выбираются позиции i_1, i_2 ; новое решение $\rightarrow S''(\overset{i_1}{1}1\overset{i_2}{0}00\dots011)$

Пример генетического алгоритма (З. Корнелюк)

Дано: $n=5$ городов, $C = \begin{pmatrix} \infty & 4 & 6 & 2 & 9 \\ 4 & \infty & 3 & 2 & 9 \\ 6 & 3 & \infty & 5 & 9 \\ 2 & 2 & 5 & \infty & 8 \\ 9 & 9 & 9 & 8 & \infty \end{pmatrix}$

$\Pi_4: 4 \ 3 \ 1 \ 2 \ 5$
 $\Pi_3: 5 \ 4 \ 3 \ 1 \ 2$
 $\Pi_2: 2 \ 1 \ 4 \ 3 \ 5$

Итерация 1: решение - перестановка; начальный популяций: $\Pi_1: 1 \ 2 \ 3 \ 4 \ 5$

$P: \begin{matrix} 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1; l_1 = 29 \\ 2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 2; l_2 = 29 \\ 5 \rightarrow 4 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 5; l_3 = 32 \\ 4 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 4; l_4 = 32 \end{matrix}$

Выбор "родителей": (Π_1, Π_2) (Π_3, Π_4)

Оператор "скрещивания" (уточненный)

$\Pi_1: \begin{array}{c|cc|cc} 1 & 2 & 3 & 4 & 5 \\ \hline 5 & 4 & 3 & 1 & 2 \end{array} \xrightarrow{i_1 \ i_2} \begin{array}{c|cc|cc} 1 & 4 & 3 & 2 & 5 \\ \hline 1 & 4 & 3 & 2 & 5 \end{array}$

(Вспомог.)

$\Pi_3: \begin{array}{c|cc|cc} 5 & 4 & 3 & 1 & 2 \\ \hline i_1 & i_2 & & & \end{array} \rightarrow 5 \ 2 \ 3 \ 4 \ 1 \xrightarrow{\text{мутация}} \begin{array}{c|cc|cc} * & 2 & 3 & * & * \\ \hline 5 & 2 & 3 & 4 & 1 \end{array} \rightarrow 5 \ 3 \ 2 \ 4 \ 1$

$\Pi_2: \begin{array}{c|cc|cc} 2 & 1 & 4 & 3 & 5 \\ \hline i_1 & i_2 & & & \end{array} \rightarrow * \ 3 \ 1 \ 2 \ * \rightarrow 4 \ 3 \ 1 \ 2 \ 5$
 $\Pi_4: \begin{array}{c|cc|cc} 4 & 3 & 1 & 2 & 5 \\ \hline i_1 & i_2 & & & \end{array} \rightarrow * \ 1 \ 4 \ 3 \ * \rightarrow 2 \ 1 \ 4 \ 3 \ 5$

Расширенный популяций: $1 \ 2 \ 3 \ 4 \ 5 \ 1 \xrightarrow{29} 1 \ 4 \ 3 \ 2 \ 5 \rightarrow 29$

Новая популяция: $P: \begin{cases} 1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 1 \\ 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 5 \\ 5 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 5 \\ 2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 2 \end{cases}$

Эволюционные стратегии

1) (μ, λ) -EA, 2) (μ, λ) -EA, 3) (μ, λ) -EA. В отличие от генетического алгоритма:

1) параметры μ, λ , которые служат показателями, означают: λ - число ро-

оператор, μ -число потомков;

2) не прилепитель оператор скремблинга

3) если оператор S_μ детерминированно выбирает μ лучших потомков из λ возможных

2 0.02

НЕ Классическая работа сс

Общая схема алгоритмов $(\mu+1)$ -EA, (μ, λ) -EA

1. Построить начальную популяцию $\Pi^0 = (S_1^0, \dots, S_n^0)$

2. Вычислить целевую ф-ию (или пригодности) $f(S_i^0)$, $i=1, n$

3. ~~Для $j=1$ до λ выполнить~~ Для $t=0$ до $t_{\max}-1$ выполнить:

3.1. Для $j=1$ до λ выполнить: ($t_{\max}$ - кол-во итераций)

3.1.1. Генерировать μ из $\{1, \dots, \mu\}$ с равномерным распределением

3.1.2. Получить решение $S_j' = \text{мутация}(S_n^t)$

3.2. ~~Нормировать~~ $\Pi^{t+1} = \begin{cases} S_\mu(S_1^t, \dots, S_n^t) \text{ для } (\mu, \lambda) \text{-EA} \end{cases}$

3.3. $t = t+1$ $\{S_n(S_1^t, \dots, S_\mu, S_1, \dots, S_n)\}$ для $(\mu+1)$ -EA

4. Получить лучшее решение из найденных

Общая схема алгоритма $(1+1)$ -EA

1. Генерировать решение S^0

2. Для $t=0$ до $(t_{\max}-1)$ выполнить:

2.1. $S' = \text{мутация}(S^t)$

2.2. Если $f(S') < f(S^t)$, то $S^{t+1} = S'$

2.3. Если $f(S') = f(S^*)$, то $S^{t+1} = S'$ с вероятностью P
иначе $S^{t+1} = S^t$

2.4 $t = t+1$

3. Получить лучшее решение алгоритма

Алгоритм имитации отжига (S. Kirkpatrick / 1983)

Алгоритм имитации отжига относится к пороговым алгоритмам. Используя

порогового алгоритма; на каждом шаге из окрестности текущего решения i_k выбирается решение j , и если разность ~~($f(i_k) - f(j)$)~~ по целевой функции между старым и новым решением не превосходит заданного порога t_k , то новое решение j заменяет текущее, в противном случае выбирается новое решение из окрестности.

Общая схема порогового алгоритма

1. Выбрать начальное решение $i_0 \in I$ и $f^* = f(i_0)$, $k=0$

2. Пока не выполнен критерий останова, выполнить:

2.1. Случайным образом выбрать j из $N(i_k)$ - окрестность i_k

2.2. Если $f(j) - f(i_k) < t_k$, то $i_{k+1} = j$ // t_k - порог

2.3. Если $f^* > f(i_k)$, то $f^* = f(i_k)$

2.4. $k = k+1$

Общая схема алгоритма имитации отжига

1. Выбрать начальное решение S и вычислить $F(S)$

2. Задать порог (температуру) T

3. Пока не выполнен критерий остановки, выполнять:

3.1. Выполнить цикл L раз:

3.1.1. Выбрать в $N(S)$ случайным образом решение S'

3.1.2. $\Delta = F(S') - F(S)$

3.1.3. Если $\Delta \leq 0$, то $S = S'$

3.1.4. Если $\Delta > 0$, то с вероятностью $e^{-\Delta/T}$ принять $S = S'$

3.2. Понижить температуру $T = T \cdot r$

Параметры алгоритма:

- начальная температура T ; - коэф. охлаждения r ; - число шагов L при заданной температуре
- критерий остановки

Алгоритмы муравьиной колонии (Дорго / М. Дорго и др.)

Характеристики элементов алгоритмов муравьиной колонии:

- компонент (фрагмент) решения - составной элемент решения задачи
- агент - рандомизированный ходный алгоритм, который итеративно строит

из множества компонент допустимое решение задачи

- эвристическая информация - это число, зависящее от решений, полученных на предыдущих итерациях, отражает желательность принятия той или иной компоненты решения

- уровень феромона - положительное число, показывающее насколько часто агентами использовались компоненты решения на предыдущих итерациях

Модель, описывающая "движения" агентов:

Модель движения - полный, связный, изометрический граф $G=(V,E)$, где V - множество вершин, соответствующих компонентам решения задачи; E - множество ребер, соответствующих связям между компонентами.

Решение задачи, построенное каждым агентом, формируется из компонент и представляет собой путь минимальной длины (если задача на min)

Компоненты решения характеризуются:

- 1 Эвристическая информация η_i (η_{ij})
- 2 Уровень феромона τ_i (τ_{ij})
- 3 Для k -го агента вероятность P_{ij}^k перехода из вершины i в j зависит от η_{ij} и τ_{ij}

Алгоритм Ant System (AS)

m - число агентов (бху); A - множество из искусственных агентов;

S_a - это решение, построенное агентом $a \in A$; α, β - параметры алгоритма, определяющие важность феромона и эвристической информации

$J(S_a \in \mathcal{L})$ - множество компонент решения, которые должны быть добавлены агентом

$a \in A$ к частично построенному решению $S_a \in \mathcal{L}$, где $\mathcal{L}$ - подмножество компонентов решения

τ_{ij} - матрица феромонов (бху); ρ, Q - $0 < \rho < 1$, $0 < Q < 1$ (бху) - участвуют в формулах изменения феромонов

Классная работа

27.02

Общая схема алгоритма Ant System

Инициализация параметров

Пока не достигнуты условия остановки, выполнять

Для всех агентов $a \in A$ выполнить:

↓ $S_a \leftarrow$ построить_решение $(\tilde{z}, \eta)$

Конец - для

↓ Модифицировать_феромон (...)

Конец - пока

Опишем приведённые алгоритме процедуры:

Построить_решение $(\tilde{z}, \eta)$ // каждый агент строит решение, добавляя его компоненты и частично построенному решению; выбор компоненты c_j происходит по следующему вероятностному правилу: $P(c_j | S_a[c_p]) = \frac{\tilde{z}_j^a \cdot \eta_j^a}{\sum_{c_k \in J(S_a[c_p])} \tilde{z}_k^a \cdot \eta_k^a}$, если $c_j \in J(S_a[c_p])$

Модифицировать_феромон (...):

$$1. \tilde{z}_i \leftarrow (1 - \rho) \cdot \tilde{z}_i + \sum_{a \in A} \Delta \tilde{z}_i^a S_a,$$

$$\Delta \tilde{z}_i^a = \begin{cases} \frac{\alpha}{f(S_a)} & \text{если компонента } c_j \text{ была использована в решении} \\ 0, & \text{иначе} \end{cases}$$

2. Замечание: при выполнении численных экспериментов с алгоритмом AS были получены следующие результаты: $f(S_a)$ - значение целевой ф-ий, частично построенное решение S_a агентом a

и такие параметры: $\alpha \rightarrow 1$, $\rho \rightarrow 5-10$

3. При проведении численных экспериментов было установлено, что AS находил лучшее решение на задачах любой размерности (тестировались на

3. Коммивояжера), трудными для AS оказались задачи большой размерности, следовательно, применяя эвристический алгоритм к решению задачи, необходимо

при проведении вычислительного эксперимента найти тестовые примеры, для которых алгоритм показывает лучшие результаты по сравнению с другими алгоритмами

Применение алгоритма AS к задаче Коммивояжера

Идея: искусственные муравьи или агенты на каждой итерации строят решение, двигаясь по графу согласно некоторому вероятностному правилу; если агент в вершине i , то ребро (i, j) добавляется в решение, т.е. компоненты решения элизируются ребра, решение — ребра

1 Для того, чтобы решение было допустимым, каждый агент имеет список запретов $(Tabulist, TL)$, в котором запоминаются уже пройденные города. Вероятность перехода из города i в j для агента k определяется:

$$P_{ij}^k = \begin{cases} \frac{[\tau_{ij}(t)]^{\alpha} [\eta_{ij}]^{\beta}}{\sum_{i \in N^k} [\tau_{ij}(t)]^{\alpha} [\eta_{ij}]^{\beta}}, & \text{если } j \in N^k \\ 0, & \text{иначе} \end{cases} \quad (1)$$

• N^k — множество вершин, которые агент k ещё не посетил;

• $\eta_{ij} = \frac{1}{d_{ij}}$, где d_{ij} — расстояние между городами i и j (вес ребра)

2 В конце каждой итерации t , после того, как все агенты построили решение, происходит пересчитывание (мультипликация) феромона: $\tilde{\tau}_{ij}(t+1) \leftarrow (1-\rho)\tilde{\tau}_{ij}(t) + \Delta\tilde{\tau}_{ij}(t)$, где ρ — коэф. испарения феромона ($0 < \rho < 1$), $\Delta\tilde{\tau}_{ij}(t)$ — суммарное изменение уровня феромона, $\Delta\tilde{\tau}_{ij}(t) = \sum_{k=1}^m \tilde{\tau}_{ij}^k(t)$, где $\tilde{\tau}_{ij}^k(t) = \begin{cases} \frac{Q}{L^k(t)}, & \text{если } (i, j) \in T^k(t) \\ 0, & \text{иначе} \end{cases}$, где $T^k(t)$ — тур, построенный k -ым агентом на итерации t ,

$L^k(t)$ — длина тура, построенная k -ым агентом на итерации t , Q : $0 < Q < 1$

Общая схема алгоритма AS для з. Коммивояжера

Инициализация параметров (феромон, матрица расстояний, α , β , ρ , Q)

Посчитать

Для агента $k \in \{1, \dots, m\}$ // построение решения

$S = \{1, \dots, N\}$ // множество заданных городов

$$TL = \emptyset$$

Выбрать город i

Повторять

Выбрать город $j \in S$ с вероятностью P_{ij}^k по формуле (1)

$$TL = TL \cup \{j\}$$

$$S = S - \{j\}$$

$$i = j$$

$$\text{Посчитать тур } T^k = T^k \cup \{i\}$$

Вычислить длину тура L^k тура T^k

До тех пор, пока $S \neq \emptyset$

Конец - для

Для всех i, j выполнять:

Использовать формулу по формуле (2)

Конец - для

До тех пор, пока не выполнен критерий остановки

Вывести тур наименьшей длины из построенных

Конец AS

Все параметры основаны на абсолютном подсчете всех проделанных

итераций и диверсификация

S'A (лимитная задача)	Итерационный	Дискретный
В.А. (геометрически активная)	Локальный поиск	Изменение T
Алгоритм муравьиной колонии	Селекция	Мутация
	Прямой изометрический перебор	Вероятностное правило выбора компонента решения

Приближенный алгоритм

Задача упаковки в контейнеры

Дано: множество предметов $L = \{1, 2, \dots, n\}$, их веса $w_i \in (0, 1)$, $i \in L$

Найти: разбиение множества L на минимальное число m подмножеств $B_1, B_2, \dots, B_m$, такое, что

$$\sum_{i \in B_j} w_i \leq 1 \text{ для всех } j: 1 \leq j \leq m. \text{ Множества } B_j - \text{контейнеры, т.е. требуется}$$

упаковать предметы w_i в минимальное число контейнеров

Данные задачи принадлежат к классу NP (сложная) и часто используется в

Математическая модель: $y_i = \begin{cases} 1, & \text{если использует } i\text{-й контейнер} \\ 0, & \text{иначе} \end{cases}$ Приближенный

$$x_{ij} = \begin{cases} 1, & \text{если предмет } i \text{ есть в контейнере } j \\ 0, & \text{иначе} \end{cases}$$

То найти $\min \sum_{i=1}^n y_i$ при условиях:

$$\begin{aligned} \sum_{i=1}^n w_i x_{ij} &\leq y_j, \quad j = \overline{1, n} \\ \sum_{j=1}^n x_{ij} &= 1, \quad i = \overline{1, n} \end{aligned}$$

Можно в качестве решения взять

предметный список в виде контейнов

$$\text{предметы: } P = \{i_1, i_2, \dots, i_n\}$$

Алгоритм случайный подгруженный (NF): предметы находятся в произвольном порядке

и упаковываются в контейнеры по следующему правилу: первый предмет помещается в

первый контейнер; ии k -мате делается попытка поместить k -й предмет в текущий

контейнер, если помещается, то помещаем в контейнер и переходим к следующему предмету;

иначе помещаем предмет в новый контейнер. Сложность: $T = O(n)$; $NF(L) \leq 2OPT(L)$

Алгоритм „Первый-подходящий“ (FF)

Предметы, которые следует упорядочить, находятся в произвольном порядке. Идея алгоритма: первый предмет помещается в первый контейнер; на k -м шаге находим контейнер с максимальной мощностью, куда помещается k -й предмет, и помещаем его туда. Если таких предметов нет, то берем новый пустой контейнер и помещаем в него.

$$T = O(n^2), \quad FF(L) \leq \left\lceil \frac{17}{10} OPT(L) + 1 \right\rceil$$

Алгоритм „Наилучший подходящий“ (BF)

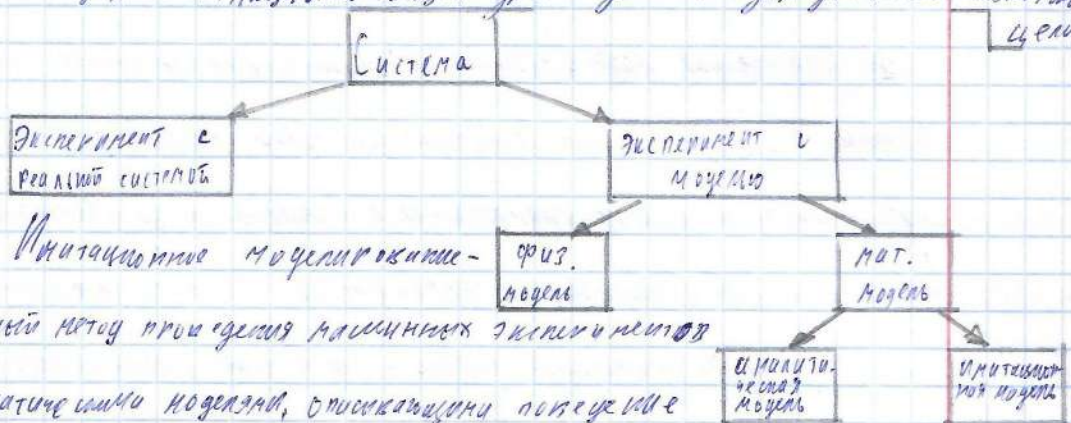
Предметы — произвольная очередь. Идея алгоритма: первый предмет $\rightarrow$ в первый контейнер; на k -м шаге размещаем k -й предмет, находим подходящую наилучшую контейнер, где для него достаточно места и объем меньше из всех возможных. Если таких нет, то берем новый контейнер и помещаем в него k -й предмет. $BF(L) = \frac{4}{3} FF(L)$

Все перечисленные алгоритмы можно изменить следующим образом: упорядочить предметы, которые следует упорядочить, например, по возрастанию весов. Такие алгоритмы — off-line алгоритмы. Алгоритмы без упорядочивания — on-line алгоритмы.

Имитационные моделирование сложных систем

Система — совокупность объектов, например, людей или механизмов, вза-

используемых и функционирующих друг с другом для достижения поставленной цели



Имитационное моделирование -

численный метод проведения машинных экспериментов

с математическими моделями, описывающими поведение

сложных систем в течение некоторого периода времени

Основное понятие из теории вероятностей

1.) Что такое эксперимент - процесс, исход которого заранее неизвестен

2.) Случайная переменная X - переменная, возможные значения которой определяются

только в результате эксперимента. Все случайные переменные имеют набор значений

ф-ца распределения случайной переменной (с.п.). $F(x) = P(X \leq x)$

3) С.п. бывают дискретные и непрерывные: непрерывные принимают значения на интервале, а дискретные принимают счетное число возможных значений

4) $P(X \in B) = \int_B f(x) dx$, $f(x)$ - ч-ца плотности распределения вероятности (если есть 0, ч-ца плотности равна 1)

5) $f(x) = F'(x)$, $x \in B$

6) Закон распределения С.п.: Н.С.П. x ; А.С.П.



крит. точка (χ^2 критой); 2-й, значимости; степени свободы: 0, 1; 0, 01; 0, 05

Существует классификация имитационных моделей:

1. Статическая модель - модель, в которой время не играет никакой роли
Пример: 710 модель, которая представляет собой сложное для вычисления мат. формулы, которые рассчитываются с помощью метода Монте-Карло (метод вычисления статистических характеристик случайных переменных)
2. Динамическая имитационная модель - время играет основную роль (модель, которая изменяется со временем)
3. Детерминированная имитационная модель - модель, которая не содержит случайных характеристик
4. Стохастическая имитационная модель - модель, которая содержит С.П. (система управления запасами; банк, больница - системы массового обслуживания)
5. Дискретная имитационная модель
6. Непрерывная имитационная модель

Дискретные, динамические и стохастические И.М называют дискретно-событийными имитационными моделями (ИМ). Пример: система массового обслуживания (СМО)

Подходы к имитационному моделированию

1. Планирование событий - написание программы работы системы на языке высокого ЯП
2. Процессорный подход - моделирование события ведется с помощью известных пакетов

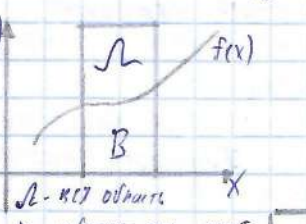
Лоту, Кейтои - имитационные моделирование, 2004 г. / 835 с.

Статистическое моделирование: метод Монте-Карло (Д. Кейтои, 1949)

Метод Монте-Карло - численный метод решения мат. задач при помощи

моделирования (М.; испытания повторяются и раз; каждое испытание независим от предыдущего; результаты всех испытаний усредняются. Через метод Монте-Карло на

примере вычислим $I = \int_a^b f(x) dx$: 1 Для вычисления интеграла генерируются точки

$f(x)$  $N_L \in L$; 2 Пусть N_B - кол-во точек под кривой $f(x)$, попавших в область B ;

3 $P(X \in B) = S_B / S_L$; $P(X \in B) = N_B / N_L$; при большом количестве точек $S_B / S_L \approx N_B / N_L$. Принцип: $I = \int_1^3 (x+1) dx$; требуется:

1 Найти оценку W этого интеграла

2 Найти: погрешность $|I - W|$ // убедиться в качестве метода;

Классная работа

13.03

- доверительный интервал.

Для доверительного интервала нужно вычислить: $S^2 = \sum_{i=1}^n \frac{(x_i - W)^2}{n-1}$

Найти доверительный интервал: $W = (b-a) \sum_{i=1}^n f(x_i) / n$

Прим: $a=1, b=3, f(x)=x+1$: $W = (3-1) \sum_{i=1}^{10} \frac{(x_i+1)}{10} = 2 \cdot \sum_{i=1}^{10} \frac{(x_i+1)}{10}$

$x_i = a + (b-a)u_i = 1 - (3-1)u_i = 1 - 2u_i$, где u_i - случайное число в $(0, 1)$

Результат испытаний

Имеем испытания	1	2	3	4	5	...	10	Из таблицы:
u_i	0,1							$\sum (x_i) = 29,834$
$x_i = 1 + 2u_i$	1,2							$\sum_{i=1}^{10} (x_i + 1) = 2 \cdot \frac{29,834}{10} =$
$f(x_i) = x_i + 1$	2,2							$= 5,968$

$$|I - w| = |6 - 5,968| = 0,032; S^2 = \frac{4}{3}$$

Доверительный интервал $(95 - \alpha)\%$, α - уровень значимости (вероятность совершить ошибку первого рода), $\alpha = 0,05$

$\bar{X} \pm Z_{\alpha} S/\sqrt{n}$: $\bar{X}$ - оценка математического ожидания

Z_{α} - критическая точка

$P = 1 - \alpha = \Phi(Z_{\alpha/2})$ - ф-я Лапласа по таблице значений $\Phi(x)$

находятся $Z_{\alpha/2}$; $Z_{\alpha/2} = 1,96$, $\alpha = 0,05$

$$\text{Доверительный интервал для оценки } w: w \pm Z_{\alpha/2} \frac{S}{\sqrt{n}} =$$

$$= 5,968 \pm 1,96 \cdot \frac{2}{\sqrt{3}} = 5,968 \pm 2,291; w \in (3,677, 8,259) \Rightarrow$$

$\Rightarrow$ вычислено верно

Моделирование системы управления запасами

Предприятие производит некоторую продукцию, запас которой хранится на складе. В начале каждого месяца предприятие пересчитывает уровень запасов и решает, какие кол-во продукции заказать у поставщика. Предприятие использует (s, S) стратегию управления запасами, где S - макс. уровень запасов, а s - критический уровень запасов. Если уровень запасов отвечает спросу на продукцию, то он немедленно удовлетворяется; если спрос превышает уровень запасов, поставка той части

продукции, которая принимает спрос как предложенный откладывается и выполняется при будущих поставках. При поступлении заказов на продукцию из резерва спроса он исполняется для максимального возможного выполнения отложенных заказов (при наличии), остаток убавляется из запаса. Имеются 3 вида затрат: - приобретение продукции; - хранение и издержки, связанные с нехваткой продукции. Требуется выбрать оптимальную стратегию управления запасами, позволяющую минимизировать затраты на содержание выполненных заказов с учетом издержек, связанных с отложенными заказами. Поскольку задача содержит случайные характеристики, для её решения применяется имитация.

Введём обозначения: - $I(t)$ - уровень запасов в момент времени t
- $I^+(t)$ - кол-во продукции, имеющейся на момент времени t
- $I^-(t)$ - кол-во продукции, поставка которой была отложена на момент времени t
- Q - объём заказа; - S - максимальный ур. запасов;
Входящие данные: S - крит. ур. заказа (точка заказа) - в этой точке достигается возможный запас
- N - число итераций; I_0 - начальный уровень запасов;
- $\dot{I}$ - затраты на заказ единицы продукции; K - затраты на заказ;
- h - затраты на хранение единицы продукции в единицу времени
- W - издержки, связанные с отложенными поставками;
- $(S_1, S_2, \dots, S_m, S_m)$ - стратегии управления запасами;

- $[\tilde{t}_1, \tilde{t}_2]$ - диапазон времени доставки продукции

Случайные переменные: - D - объем спроса на запас;

- T_D - пролежит ли между возникновением спроса // непрерывная С.П. и доставкой С.П.
- $\tilde{t}$ - время доставки продукции на некотором интервале, $\tilde{t} \in [\tilde{t}_1, \tilde{t}_2]$

Переменные состояния: $I(t)$ и Q

События (действия, которые изменяют состояние системы в какое-то момент времени):

- возникновение спроса на продукцию;
- поступление заказа на продукцию; - отгрузка заказов

03.04

Классная работа

Рассмотрим событие „возникновение спроса на продукцию“:

1. Генерация спроса
2. Уменьшение уровня запасов на объем спроса
3. Планирование следующего возникновения спроса

Рассмотрим событие „поступление заказа на продукцию“:

1. Поступление заказов
2. Увеличение уровня запасов на заказанный ранее количество

Рассмотрим событие „отгрузка заказов продукции“:

- если $I(t) < S$, то - определение количества заказанного товара $S - I(t)$

- определение затрат на приобретение заказов

- планирование поступлений заказов

иначе: - Планирование следующей оценки запасов

Выходные данные:

- $(\pi_1, \dots, \pi_n)$ - средние затраты на обслуживание заказов на стратегиях
- $(\chi_1, \dots, \chi_n)$ - средние затраты на хранение продукции для стратегий $(s_1, s_1), \dots, (s_n, s_n)$
- $(c_1, \dots, c_n)$ - средние издержки, связанные с ошибочной продукцией для стратегий
- $(c_{об1}, \dots, c_{обn})$ - средние общие затраты для стратегий $(s_1, s_1), \dots, (s_n, s_n)$
- (s^*, s^*) - стратегия с наименьшей общими затратами $S_{об}$ в единицу времени

Вычисление оценок имитационной модели (показатели эффективности моделируемого процесса управления запасами)

1. Среднее кол-во продукции, находящейся в запасах в течение N единиц времени:
$$\bar{I}^+ = \frac{\int_0^N I^+(t) dt}{N}$$
2. Средние затраты на хранение: $C_x = h \cdot \bar{I}^+$
3. Среднее кол-во товара в отложенных поставках в течение времени t : $\bar{I}^- = \frac{\int_0^N I^-(t) dt}{N}$
4. Средние издержки C_0 , образовавшиеся в связи с отложенными поставками: $C_0 = w \cdot \bar{I}^-$
5. Объем заказа: $Q = \begin{cases} S - I, & \text{если } I < S \\ 0, & \text{если } I \geq S \end{cases}$
6. Если запас пополняется на Q единиц, то затраты $C_n = K + i \cdot Q$, где K - затраты на заказ, i - затраты на заказ единицы продукции (при $Q=0$ нет затрат)
7. Средние общие затраты $C_{об} = C_n + C_x + C_0$

Общая схема алгоритма

1. Генерация случайных переменных (π) : объем спроса D , принятого времени

между возникновением спроса T_0

2 Имитация управления запасами для (s_1, s_2) // время тиасов

- если $I(t) < s$, то репарировать $\tau \in [\tau_1, \tau_2]$ // время доставки

- увеличить Q на $s - I(t) \longrightarrow$ находить b , если "

- затраты C_n на заказ увеличить на $K + iQ \longrightarrow$

- если $t \in T_b$, то репарировать спрос D :

- уменьшить уровень запасов $I(t)$ на D

- если $Q > 0$ и $t \geq \tau$, то увеличить $I(t)$ на Q // поступили запас

3 Подсчет затрат для стратегии (s_j, s_j) :

Затраты на хранение C_x , издержки на ^{исполнение} ~~исполнение~~ C_d , C_n , C_{ob}

4 Выбор лучшей стратегии из имеющихся: (s^*, s^*) по критерию

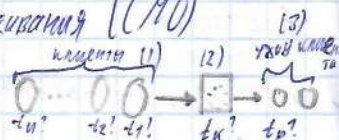
минимизации средних общих затрат

Пример отформатированных результатов моделирования

Стратегия (s_1, s_2)	Средние общие затраты	Средние затраты на приобретение товара	Средние затраты на хранение	Средние издержки, связанные с поломкой
(15, 50)	310,034	143,61	6,16	150,32

Моделирование системы массового обслуживания (МО)

(МО с одним устройством: состоит:



1) клиенты (требования, заявки); 2) устройство обслуживания

• $t_1, t_2, \dots, t_n$ - случайный поток событий (приход клиентов) // очередь FIFO

• $\square$ - Устройство обслуживания: обслуживает t_k ?

- Число клиентов - это событие, приход клиента - это событие

Модели потока поступления клиентов в СМО

1. Простейший поток - пуассоновский (1): Поток событий - последовательность событий, происходящих один за другим в какие-то моменты времени t_1, t_2, t_n . Пример: вызовы ^{на} телефонной станции. Вывод потока:

- 1) регулярный - события следуют один за другим через строго определенные промежутки времени (очень редко встречается в реальных системах)
- 2) стационарный / однородный -  - вероятность поступления того или иного числа событий на интервал времени T зависит только от длины интервала и не зависит от его положения (редко встречается в реальных системах)

3) Поток событий без последствий: для любых непересекающихся интервалов времени число событий, попадающих в один, не зависит от числа событий, попадающих на другой.



4) Огудинакий поток - практическая невозможность поступления двух или более событий в короткий промежуток времени

Мат. ожидание $\lambda > 0$ числа требований, поступающих в единицу времени, называется интенсивностью потока. Для стационарного эта величина постоянна, т.е. не зависит от времени. Вывод: простейший поток - это модель притока клиентов в СМО для малого промежутка времени (не применяется в реальных системах)

2 Пуассоновские процессы - это случайные процессы:

- 1) однородные / стационарные
- 2) неоднородные / нестационарные

Развивающийся во времени процесс, характеристики которого в любой момент времени t С.П., называется случайным

Случайный процесс $\{N(t), t \geq 0\}$ называется процессом поступления, где $N(t) = \max \{i, t_i \leq t\}$; $N(t)$ - число клиентов, поступивших в систему в момент времени t или до момента времени t ; t_i - время поступления клиента i

0.04

Классная работа

Однородный пуассоновский процесс (модель прихода клиентов в течение некоторого промежутка времени в СМО):

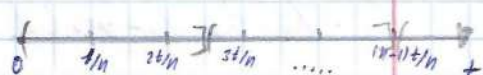
• $N(t)$ - кол-во клиентов в момент времени t

Случайный процесс $\{N(t), t \geq 0\}$ - однородный пуассоновский, если:

- 1) $N(0) = 0$
- 2) Кол-во клиентов в $(t, t+\Delta t]$ не зависит от кол-ва клиентов в $[0, t]$
- 3) $\Delta t \rightarrow 0$, $P(N(\Delta t) = 1) \approx \lambda \Delta t$, где λ - интенсивность; клиенты поступают в устройство по отдельности
- 4) $\Delta t \rightarrow 0$, $P(N(\Delta t) \geq 2) = 0$

Пример: пусть интенсивность двуканальной работы СМО; установив, что в большинстве систем интенсивность поступления клиентов из этого интервала будет практически постоянным и процессом можно описать процесс будет моделью притока клиентов в течение этого промежутка времени (возбуждения описывающего процесс);

1. Число событий (притока клиентов), происходящих в интервале времени t - случайная С.П. со средним λt , где λ - интенсивность событий в единицу времени. Схематичное изображение:



= разбиён $[0, t]$ на n непересекающихся интервалов длиной t/n .
 = разбиён $[0, t]$ на пересекающихся интервалов.

Если в интервал попал хотя бы один клиент, то это успех, иначе - неудача. Поскольку заранее неизвестно сколько точек попадёт в каждый из интервалов, то число таких точек - это С.П., подчинённая биномиальному закону распределения. Вероятность появления k успехов: $p = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$, где (1)
 p - вероятность успеха, $(1-p)$ - вероятность неудачи. ($p = \lambda t/n = \lambda t/n$)

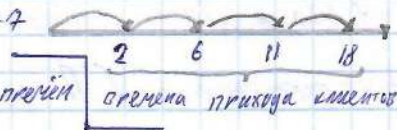
Поскольку в реальных СМО кол-во клиентов очень большое, рассмотрим, что будет, если $n \rightarrow \infty$. $\lim_{n \rightarrow \infty} \text{Binomial}(n, k) = \lim_{n \rightarrow \infty} \frac{n!}{k!(n-k)!} \left(\frac{\lambda t}{n}\right)^k \left(1 - \frac{\lambda t}{n}\right)^{n-k} =$
 $= \frac{(\lambda t)^k}{k!} e^{-\lambda t} \rightarrow \text{формула Пуассона. } P_k(\lambda t) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}$ (2) - вероятность k успехов при $n \rightarrow \infty$
 Т.е. при $n \rightarrow \infty$ биномиальное распределение переходит в пуассоновское

Таким образом, число событий, происходящих в интервале времени, - число

исходя С.П. со средним λt , потоку процесс - Пуассоновский.

2. Интервальные времена к СМО. Пусть X_1 - время прихода 1 клиента, X_n - время прихода клиента между $(n-1)$ клиентом и n событием ($n \geq 1$). $\{X_n\}, n=1,2,\dots$ - последовательность интервальных времён. Поскольку известно время прихода клиентов, то интервальные времена - это С.П.

Пример: $X_1=2, X_2=4, X_3=3, X_4=7$



Найдём распределение интервальных времён

• X_1 : т.к. X_1 - это С.П., она имеет функцию распределения $F(x_1) = P(X_1 \leq t)$.

т.к. между временами нет событий, то $F(x_1) = P(X_1 > t) = P(N(t)=0) = \frac{(\lambda t)^{N(t)}}{N(t)!} e^{-\lambda t} = \frac{(\lambda t)^0}{0!} e^{-\lambda t} = e^{-\lambda t}$; таким образом,

X_1 имеет экспоненциальное/показательное распределение

• Аналогично для $X_2, X_3, \dots$

Интервальные времена $X_1, X_2, \dots$ являются независимыми и распределёнными С.П.; закон распределения - экспоненциальный

Алгоритм генерации случайного пуассоновского процесса

Цель: для генерации этого процесса сначала нужно сгенерировать сл. числа $U_1, U_2, \dots, U_n$, равномерно распределённые на $(0,1)$; далее сгенерировать интервальные времена $X_i = -\frac{1}{\lambda} \log U_i$



Вход: T

Выход: $\bullet I$ - число событий, которые произошли к моменту времени T

• $S(1), \dots, S(I)$ - времена событий (приходов клиентов) в восстановительном порядке

1. $t=0, I=0$

2. $U = \text{uniform}(0, 1)$

3. $X = \frac{-1}{\lambda} \log U$ // интервалы между событиями 

4. $t = t + X$

5. Если $t > T$, то: конец

6. иначе: $I = I + 1, S(I) = t$

7. Идти на "Шаг 2"

Пуассоновский неоднородный процесс

Стохастический процесс $N(t), t \geq 0$ будет неоднородным пуассоновским, если

1) $N(0) = 0$

2) $[t, t+h]$ не зависит от $(0, t]$

3) $P(N(t+h) - N(t) = 1) \approx \lambda(t) \cdot h; h \rightarrow 0$. $\lambda(t)$ - ф.п. интенсивности, зависящая от t

4) $P(N(t+h) - N(t) \geq 2) = 0$



За метками:

1) Среднее число событий, которое произошло за время t : $m(t) = \int_0^t \lambda(s) ds, t \geq 0$

Свойства неоднородного Пуассоновского процесса:

1. Такие же, как и 1 свойства однородного пуассоновского процесса

2. Закон распределения интервалов между событиями не экспоненциальный. (Характерное для неоднородного процесса)



• T - с.п., т.к. t_0 и t_0+S - с.п.

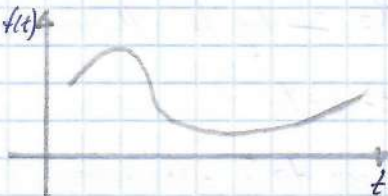
Напишем Ф-ию распределения для интервала T , когда в нём нет клиентов: $F(t) = P(T > t) = e^{-\lambda(t)} = e^{-\int_{t_0}^{t_0+t} \lambda(t) dt}$

$$f(t) = F'(t) = \lambda(t_0 + t) e^{-\int_{t_0}^{t_0+t} \lambda(t) dt} \quad (t > 0) - \text{не эквивалентное распределение}$$

Пример: $t_0 = 0, \lambda = 0, b = 2$ $f(t)$

$$\lambda(t) = a + bt$$

$$f(t) = (a + b(t_0 + t)) e^{-a + bt_0 - \frac{bt^2}{2}}$$



Генерация неоднородного пуассоновского процесса

$\lambda > 0$, $\lambda(t) \leq \lambda$ для всех $t \in [0, T]$; $\lambda(t) = \lambda \cdot p(t)$; $p(t) = \frac{\lambda(t)}{\lambda}$

1. $t = 0$

2. Генерировать $U_1 \sim \text{uniform}(0, 1)$

3. $X = -\frac{1}{\lambda} \log U_1$

4. $t = t + X$

5. Генерировать $U_2 \sim \text{uniform}(0, 1)$

6. Если $U_2 \leq \frac{\lambda(t)}{\lambda}$, то $T_s = t$, конец

7. Идти на „Шаг 2“

Моделирование системы управления запасами

В магазине имеется запас определенного вида товара, который он продает по цене v за единицу. Для того, чтобы удовлетворять покупателей, магазин должен иметь определенный уровень запаса. Каждый раз, когда запаса становится меньше, то он заказывает y единиц товара. Известна цена